Secure Text Steganography Using Unicode Embedding and Encryption

Client-Side Steganographic Encryption System

23BAI1188 M. Reddy Sasi Kiran

23BAI1222 V. Jawahar Raju

23BAI1187 C.R. Suhas

Prof. Radhika Selvamani

B.Tech — CSE | Scope

Dept. of Computer Science & Engineering | School of Computing Sciences & Engineering

Introduction

Where Can This Be Applied?

Whistleblower Comms

Journalists and activists embed secrets in plain social media posts — no suspicious encrypted files needed.

Digital Watermarking

Publishers hide invisible ownership signatures inside distributed text to track leaks and piracy.

Covert Command Channels

Security researchers test covert C2 channels inside normal-looking web traffic or chat messages.

Stealth Evidence Tagging

Legal or forensic teams embed invisible metadata inside document copies to trace unauthorized sharing.

Restricted-Environment Messaging

In censored or monitored networks, users carry hidden payloads inside innocuous public text.

Secure QR / NFC Payloads

Invisible encrypted annotations embedded into visible product descriptions for tamper-proof verification.

Literature Review

Naive Invisible-Text Embedding

Existing methods

1. Convert hidden text to UTF-8 bytes
2. Map bits to 2 invisible chars: 0 → \u200b, 1 → \u200c
3. No encryption, no key, no salt
4. Append invisible block to END of message
5. Output: hello world\u200b\u200c\u200b...

Why existing methods wont work

Trailing Trimming
Platforms strip whitespace & invisible chars at end of text — entire payload gone.

Unicode Normalization
NFC/NFKC normalization collapses or removes redundant invisible characters.

Character Filtering
Repeated \u200b blocks look like noise — bulk-removed by messaging pipelines.

Pattern Detection
One continuous invisible block at a predictable position is trivial to detect and strip.

No Recovery Safety
Even partial removal outputs garbage — no integrity check to catch corruption.

Problem Statement

What needs to be solved

P1

No Confidentiality in Naive Stego

Existing approaches embed raw bits — any observer who extracts invisible chars reads the secret directly. No cryptographic protection exists.

P2

Platform-Level Stripping

WhatsApp, Gmail, and Google Docs normalize Unicode and trim trailing invisibles, destroying naively embedded payloads during transit.

P3

Absence of Integrity Verification

Without authentication, single-character corruption produces silent garbage output — receiver cannot detect tampering or wrong passwords.

P4

Predictable Embedding Position

Trailing placement and repeated identical invisible symbols form detectable statistical patterns, making the presence of hidden data obvious.

P5

Weak Key Management

Direct use of user-supplied passwords as keys is trivially brute-forceable; no salt, no hardening, no key-stretching is applied.

Analysis — Encryption Pipeline

Step-by-step transformation of secret text into carrier

ANALYSIS

Secret Text *Input string*	**UTF-8 Bytes** *bytes.js*	**Compress** *compress.js*	**Bitwise NOT** *compliment.js*	**Salt (8B)** *crypto.random*	**PBKDF2** *pbkdf2.js*
Embed *embed.js*	**Shrink** *shrink.js*	**ZWC Encode** *encode.js*	**Payload Build** *salt+hmac+cipher*	**HMAC-SHA256** *hmac.js*	**AES-256-CTR** *aesCtr.js*

Full flow: Secret Text → UTF-8 → Compress → Bitwise NOT → Salt → PBKDF2 → AES-CTR → HMAC → Payload → ZWC Encode → Shrink → Embed → Carrier

Analysis — Encryption: Steps 1–6

From raw text to encrypted ciphertext

ANALYSIS

1 String to UTF-8 Bytes

In: "hello vit" **Out:** [104,101,108,108,111,32,118,105,116]

Process: TextEncoder converts text to byte array

Why: Cryptographic functions operate on bytes, not strings

2 Compression (deflate)

In: UTF-8 byte array **Out:** Compressed byte array

Process: CompressionStream (deflate) if supported; else raw bytes

Why: Reduces payload size — smaller = fewer invisible characters needed

3 Bitwise NOT Transformation

In: Compressed bytes **Out:** Prepared bytes (transformed)

Process: Each byte: b → ~b (flip all bits)

Why: Extra obfuscation layer before encryption; reversed on reveal

4 Salt Generation

In: None **Out:** salt (8 bytes)

Process: crypto.getRandomValues() generates 8 random bytes

Why: Ensures PBKDF2 output is unique each run; prevents precomputed attacks

5 PBKDF2 Key Derivation

In: password + salt **Out:** IV (first 16B) + AES key (next 32B)

Process: PBKDF2 with SHA-512, 10,000 iterations → 48 derived bytes

Why: Turns weak 5-digit password into strong crypto material; same inputs = same key

6 AES-256-CTR Encryption

In: Prepared bytes + AES key + IV **Out:** Ciphertext (same length as input)

Process: Generate keystream from (IV + counter), XOR with plaintext

Why: No padding needed; stream-friendly; length-preserving for ZWC encoding

Analysis — Encryption: Steps 7–12

From HMAC to final embedded carrier

ANALYSIS

7 HMAC-SHA256 (Integrity)

In: Prepared bytes + AES key **Out:** 32-byte authentication tag

Process: HMAC-SHA256(key, prepared_bytes)

Why: Detects wrong password, tampering, or corruption at decryption time

8 Payload Construction

In: salt + hmac + ciphertext **Out:** payload_bytes (structured binary)

Process: Concatenate in fixed order

Why: Bundles all recovery data; salt+hmac+cipher split is known at both ends

9 Byte → 2-bit Splitting

In: payload_bytes **Out:** 2-bit stream (4× length of payload)

Process: Each byte → 4 chunks of 2 bits (MSB-first)

Why: ZWC alphabet has 4 symbols; 2-bit chunks map 1-to-1 to each symbol

10 2-bit → Zero-Width Chars

In: 2-bit chunks **Out:** Invisible Unicode character stream

Process: 00→\u200c 01→\u200d 10→\u2061 11→\u2062

Why: Valid Unicode invisibles survive most text pipelines; look like normal text

11 Shrink Optimization

In: ZWC stream **Out:** Shorter, less-repetitive invisible stream

Process: Detect repeat pairs; replace with \u2063 / \u2064 tokens

Why: Reduces payload size and breaks obvious repetition patterns

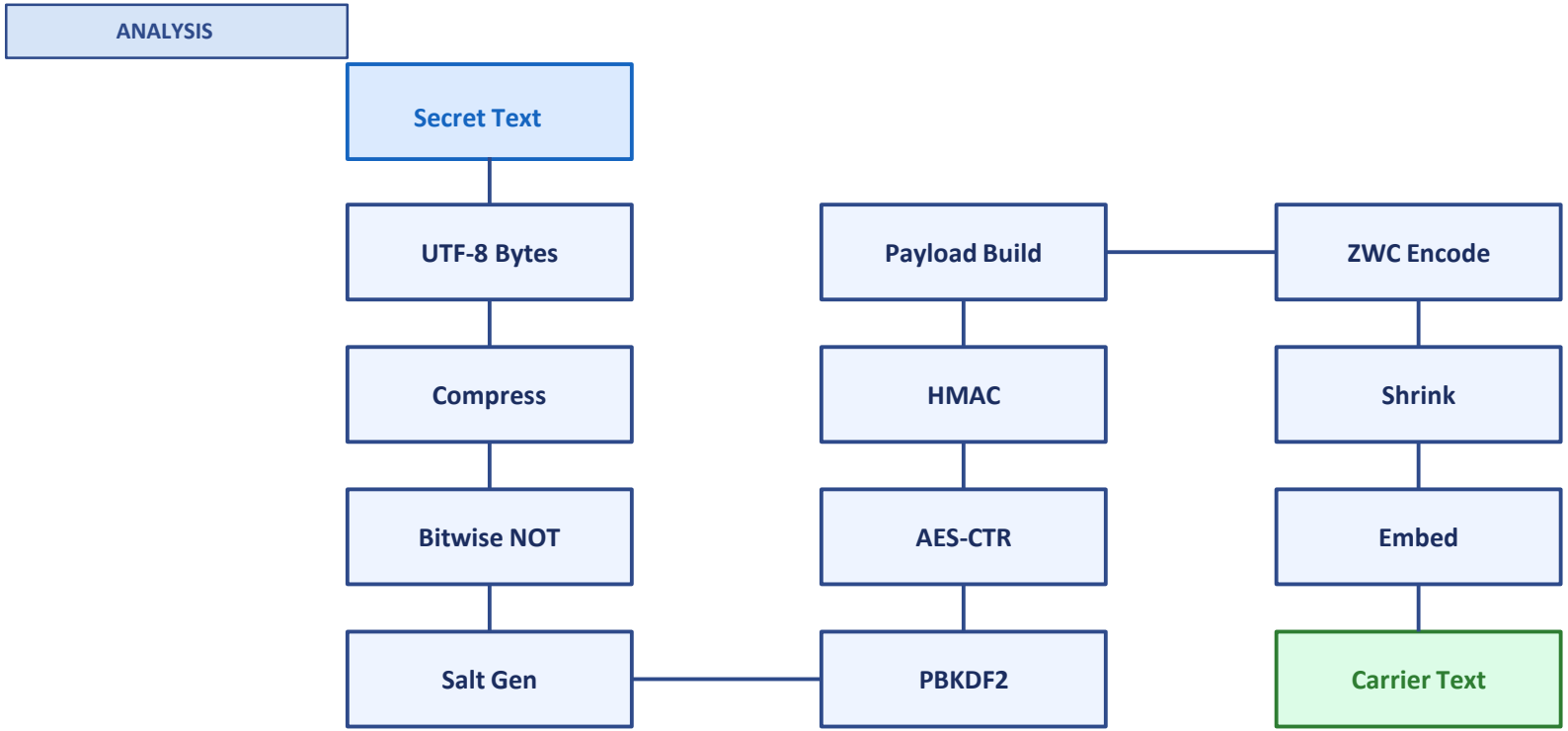
12 Embed into Cover Text

In: Cover text + invisible stream **Out:** Carrier text (looks normal, contains hidden payload)

Process: Split words; pick random index in first half; inject invisible stream

Why: Non-trailing placement survives trimming; blends naturally into visible text

Analysis — Encryption Flow Diagram



Analysis — Decryption Pipeline

Step-by-step recovery from carrier to secret

1

Extract Invisible Stream

In: Carrier text **Process:** Scan words for ZWC chars; extract invisible prefix

Out: invisible_stream **Why:** Payload lives only in invisible characters; visible text is ignored

2

Expand (Reverse Shrink)

In: invisible_stream **Process:** Read compression flag; restore shortened token pairs

Out: original_zwc_stream **Why:** Must undo shrink before decoding; flag in first char identifies pairs

3

ZWC → Bytes

In: zwc_stream **Process:** Every 4 chars → 1 byte via 2-bit reverse mapping

Out: payload_bytes **Why:** Reconstructs exact binary payload ready for splitting

4

Split Payload

In: payload_bytes **Process:** salt = [0:8] hmac = [8:40] ciphertext = [40:]

Out: salt, hmac, ciphertext **Why:** Fixed structure set during encryption; enables targeted extraction

5

PBKDF2 (same params)

In: password + extracted salt **Process:** SHA-512, 10,000 iter → 48B; split into IV + AES key

Out: IV (16B) + AES key (32B) **Why:** Deterministic: same inputs → same key/IV; enables correct decryption

6

AES-256-CTR Decrypt

In: ciphertext + key + IV **Process:** Regenerate same keystream; XOR with ciphertext

Out: prepared_bytes (pre-encryption state) **Why:** CTR decryption = re-encryption; inverse is identical operation

7

HMAC Verification

In: prepared_bytes + key + stored hmac **Process:** Recompute HMAC; constant-time compare with stored

Out: Valid → continue | Mismatch → reject **Why:** Guards against wrong password, tampering, or platform corruption

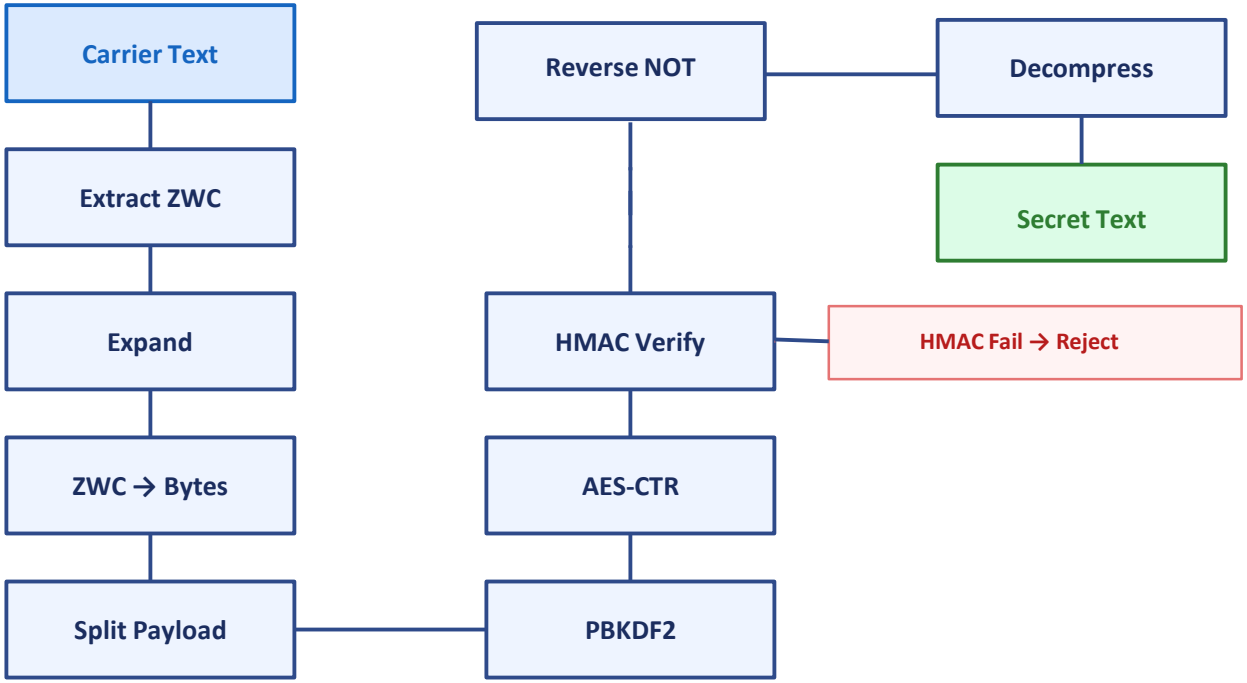
8

Reverse NOT + Decompress

In: prepared_bytes (if valid) **Process:** ~b on each byte, then deflate decompress back to string

Out: Original secret text **Why:** Reverses preprocessing; final step restores human-readable message

Analysis — Decryption Flow Diagram



Conclusion

CONCLUSION

01

End-to-End Client-Side Security

No server involvement — encryption, key derivation, and embedding run entirely in the browser via Web Crypto API, eliminating server-side attack surface.

02

Layered Defense Pipeline

Compression, bitwise NOT, PBKDF2 key hardening, AES-256-CTR encryption, and HMAC-SHA256 integrity form a multi-layer defense uncommon in student-level cryptography projects.

03

Non-Trivial Steganography

Payload is embedded inside sentence word boundaries using a structured ZWC stream — not appended at trailing positions — making it more resilient against platform-level sanitization.

04

Safe-Failure Guarantee

HMAC verification with constant-time comparison ensures the system never outputs incorrect plaintext; any wrong password or corruption produces a controlled rejection.

05

Acknowledged Limitations

5-digit password space is intentionally constrained for demo purposes; aggressive Unicode normalization can still strip invisible characters on some platforms.